

北京邮电大学信息与知识获取课程实验报告

——信息抽取系统

小组成员：赵宇鹏

学号：2022211119

班级：2022211304

北京邮电大学信息与知识获取课程实验报告——信息抽取系统

1. 作业要求
 - 1.1 基本要求
 - 1.2 扩展要求
2. 项目介绍
 - 2.1 项目结构
 - 2.2 数据获取
 - 2.3 文件概述
 - 2.3.1 load.py
 - 2.3.2 process.py
 - 2.3.3 main.py
 - 2.4 创新以及算法优化
 - 2.4.1 多媒体实现
 - 2.4.2 人工评估反馈
 - 2.4.3 信息抽取优化
3. 结果展示
4. 实验总结

1. 作业要求

1.1 基本要求

设计并实现一个信息抽取实验系统，中、英文皆可，可以在作业2信息检索系统的基础上实现，也可以单独实现。特定领域语料根据自己的兴趣选定，规模不低于100篇文档，进行本地存储。对自己感兴趣的特定信息点进行抽取，并将结果展示出来。其中，特定信息点的个数不低于5个。可以调用开源的中英文自然语言处理基本模块，如分句、分词、命名实体识别、句法分析。信息抽取算法可以根据自己的兴趣选择，至少实现正则表达式匹配算法的特定信息点抽取。最好能对抽取结果的准确率进行人工评价。界面不作强制要求，可以是命令行，也可以是可操作的界面。提交作业报告和源代码。鼓励有兴趣和有能力的同学积极尝试优化各模块算法，也可关注各类相关竞赛。

1.2 扩展要求

鼓励有兴趣和有能力的同学积极尝试多媒体信息抽取以及优化各模块算法，也可关注各类相关竞赛。自主开展相关文献调研与分析，完成算法评估、优化、论证创新点的过程。

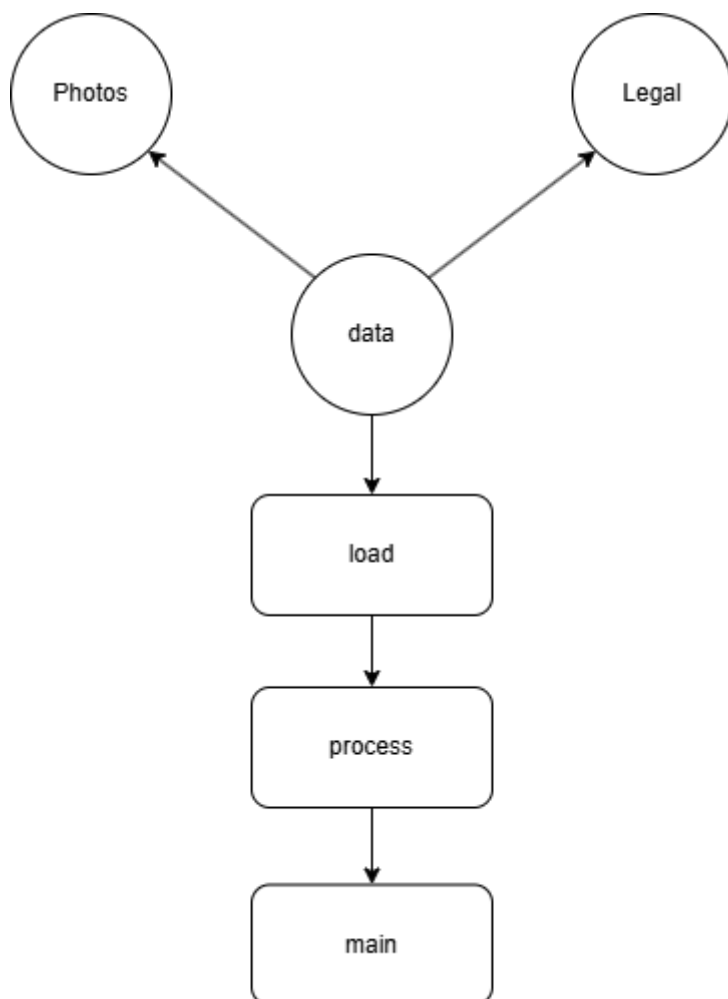
2. 项目介绍

2.1 项目结构

2.2 数据获取

本项目的数据集，全部都由BBC网站上爬取下来，分为 `articles`、`photos`、`videos` 三部分，其中 `articles` 包含98篇BBC新闻，`videos` 包含64篇新闻，由于在爬取时视频无法储存，所以将含视频的新闻放到此类，`photos` 则包含5张图片，对应 `articles` 中的5篇新闻，以作多媒体示例。

2.3 文件概述



本项目由三个python文件组成，分别是：

- `load.py`：负责数据加载。
- `process.py`：负责进行分词分句，命名实体的识别以及相关信息点的抽取。
- `main.py`：整个项目的入口，负责实现可视化以及调用其他模块的函数来执行整个项目。

2.3.1 load.py

`load.py` 的作用是加载已经准备好的相关文档和图片，在这里我准备的文档主要是法律判决书文件以及学术论文图片，这个文件的实现比较简单，主要是依靠 `pdfplumber` 库的方法来对PDF文本信息进行识别和读取，然后删除所有换行符以及空格，关键代码如下：

```
1 with pdfplumber.open(file_path) as pdf:
2     # 提取所有页面文本
3     for page in pdf.pages:
4         text = page.extract_text()
5         if text:
6             full_text += text + " "
7
```

```

8         # 1. 替换所有换行符为空字符
9         cleaned_text = full_text.replace('\n', '')
10        # 2. 替换多个连续空格为单个空格
11        cleaned_text = re.sub(r'\s+', ' ', cleaned_text)
12        # 3. 替换中文间的空格（两个中文字符之间的空格）
13        cleaned_text = re.sub(r'([\u4e00-\u9fa5])\s+([\u4e00-\u9fa5])',
r'\1\2', cleaned_text)
14        # 4. 去除首尾空格
15        cleaned_text = cleaned_text.strip()
16
17        return {
18            'id': Path(file_path).stem,
19            'filename': Path(file_path).name,
20            'content': cleaned_text,
21            'file_path': str(file_path),
22            'language': 'chinese',
23            'document_type': 'legal'
24        }

```

2.3.2 process.py

本文件是项目的主体部分，关键的信息抽取都在本文件实现，接下来具体介绍一下相关思路 and 实现：

首先我使用的是斯坦福的开源自然语言处理通用处理器 `Stanza`，以下是初始化参数代码：

```

1  # 初始化Stanza管道
2  # Args:
3  # language: 处理语言, 'zh'表示中文
4  # use_gpu: 是否使用GPU加速
5  # download_if_missing: 如果模型不存在是否自动下载
6  # batch_size: 批处理大小
7  self.nlp = stanza.Pipeline(
8      lang=language,
9      processors=processors,
10     use_gpu=(self.device == "cuda"),
11     download_if_missing=download_if_missing,
12     ner_batch_size=self.batch_size,
13     logging_level='INFO'
14 )

```

我们使用 `Stanza` 的目的主要是获得文档的分句，分词，词性以及命名实体来为后续的抽取做准备，最终得到的分析结果格式如下：

```

1  return {
2      "sentences": sentences,
3      "tokens": tokens,
4      "pos_tags": pos_tags,
5      "entities": entities,
6      "raw_doc": doc,
7      "processing_time": processing_time
8  }

```

经过测试，我发现无论是单独使用正则表达式进行抽取，还是通过命名实体进行抽取，最终得到的信息点抽取结果都不是特别好，由于我准备的法律信息文档的格式非常类似，所以在这里我最终换成了针对法律信息的正则表达式加上命名实体一起进行抽取的方法来实现信息抽取工作，首先根据文档准备一个粗略的关键词集合以及通用正则表达式，由于我们要抽取的是法律判决书，所以抽取的特定信息点应该为：

1. 案件基本信息（案号、法院、案件类型） -> 案件标识
2. 当事人信息（原告、被告、代理人） -> 案件主体
3. 法律依据（引用的法律条文） -> 法律基础
4. 执行情况（执行结果、执行措施） -> 案件结果
5. 时间节点（申请时间、裁定时间） -> 时间线

根据以上的特定信息点，我们就可以准备好正则匹配需要的关键词以及相应的正则表达式：

```
1 # 法律角色关键词及其相似词
2 self.legal_roles = {
3     "原告": ["原告", "起诉方", "申请人", "申请执行人", "权利人", "权利方"],
4     "被告": ["被告", "被诉方", "被执行人", "被申请人", "义务方", "当事人"],
5     "代理人": ["委托代理人", "代理人", "律师", "辩护人", "诉讼代理人"],
6     "法官": ["审判长", "审判员", "法官", "主审法官", "合议庭成员"],
7     "书记员": ["书记员", "记录员", "法庭记录员"],
8     "执行长": ["执行长"],
9     "执行员": ["执行员", "执行法官", "执行人员"]
10 }
11 # 案件类型关键词
12 self.case_types = {
13     "民事": ["民事", "民初", "民终", "民执"],
14     "刑事": ["刑事", "刑初", "刑终"],
15     "行政": ["行政", "行初", "行终"],
16     "执行": ["执行", "执", "执恢"]
17 }
18 # 编译常用正则表达式
19 self.case_number_pattern = re.compile(r'[(\d{4})]\)[^\)]+[字第\d+号]')
20 self.date_pattern = re.compile(r'(\d{4}年\d{1,2}月\d{1,2}日|\d{1,2}月\d{1,2}日)')
21 self.money_pattern = re.compile(r'【壹贰叁肆伍陆柒捌玖拾佰仟万亿万零元整\d+元】')
22 self.law_article_pattern = re.compile(r'《([^\]]+ )》【^第】*第【零一二三四五六七八九十百千+】条】')
23 # 特定角色模式
24 self.plaintiff_pattern = re.compile(r'申请执行人[: :]\s*([\u4e00-\u9fa5]{2,3})[\s, .]')
25 self.defendant_pattern = re.compile(r'被执行人[: :]\s*([\u4e00-\u9fa5]{2,3})[\s, .]')
26 self.rights_holder_pattern = re.compile(r'权利人\s*([\u4e00-\u9fa5]{2,3})')
27 # 法律人员模式
28 self.execution_personnel_pattern = re.compile(r'执行长\s*([\u4e00-\u9fa5]{2,3})\s*执行员\s*([\u4e00-\u9fa5]{2,3})\s*执行员\s*([\u4e00-\u9fa5]{2,3})')
29 self.clerk_pattern = re.compile(r'书记员\s*([\u4e00-\u9fa5]{2,3})')
```

接下来就可以进行信息抽取工作了，具体的工作流程如下：

```
1 # 先进行基础NLP处理
2 nlp_result = self.process(text)
```

```

3 # 提取关键信息点
4 legal_info = {
5     "case_info": self._extract_case_info(text),
6     "parties": self._extract_parties_info(text, nlp_result),
7     "legal_basis": self._extract_legal_basis(text),
8     "execution": self._extract_execution_info(text),
9     "timeline": self._extract_timeline(text),
10    "court": self._extract_court_name(text),
11    "legal_personnel": self._extract_legal_personnel(text, nlp_result)
12 }
13 # 构建关系描述
14 legal_info["causal_chain"] = self._build_causal_chain(legal_info)
15 return legal_info

```

首先进行基础的NLP分词分句以及命名实体的抽取，然后分别调用抽取函数抽取相应的信息点，最后构造关系描述进行信息的总结和串联，接下来以当事人信息抽取函数 `_extract_parties_info()` 以及 `_build_causal_chain()` 为例进行介绍和讲解：

`_extract_parties_info()`：

该函数的作用是提取当事人信息：原告，被告，代理人等，由于在实际测试中，我发现原告被告有可能是人，也有可能是公司等机构，所以需要进行额外处理，首先利用宽松的正则匹配获取相应文本的位置，然后对初步匹配的文本，检查是否是机构的名称，若是则查找后续的边界分隔符，并获取机构的名称，若不是机构，则采用2-4字的正则匹配来匹配人名，若正则匹配失效，没有找到当事人，则使用命名实体，提取 PERSON、ORG、GPE 这几个命名实体，再通过查找命名实体的上下文来判断该实体的角色，从而实现对当事人的识别，主体代码如下：

```

1 # 先用宽松正则获取位置
2 rough_match = re.search(r'申请执行人[: :]\s*([\u4e00-\u9fa5]+)(?:[, ,。 \. : \s] | $)', text[plaintiff_pos:plaintiff_pos+200])
3 if rough_match:
4     # 初步匹配的文本
5     potential_name = rough_match.group(1).strip()
6     # 检查是否可能是机构名称
7     is_org = any(keyword in potential_name for keyword in org_keywords)
8     # 根据是否为机构名采取不同的处理策略
9     if is_org:
10        # 尝试找到机构名的边界（如逗号、句号等）
11        end_markers = [",", "。", ":", ":", " ", "\n", ">", ">", ">", "\n"]
12        end_pos = len(potential_name)
13        if len(plaintiff_name) > 3 and len(plaintiff_name) <= 30:
14            parties.append({
15                "name": plaintiff_name,
16                "role": "原告",
17                "type": "organization",
18                "details": {}
19            })
20            identified_names.add(plaintiff_name)
21        else:
22            # 对于个人名字，采用严格的2-4字符限制
23            person_match = re.search(r'申请执行人[: :]\s*([\u4e00-\u9fa5]{2,4})(?:[, ,。 \. : \s] | $)', text[plaintiff_pos:plaintiff_pos+50])
24            if person_match:
25                plaintiff_name = person_match.group(1).strip()
26                details = self._extract_person_details(plaintiff_name, text)

```

```

27     parties.append({
28         "name": plaintiff_name,
29         "role": "原告",
30         "type": "person",
31         "details": details
32     })
33     identified_names.add(plaintiff_name)
34
35     # 如果上述方法未找到当事人，结合NER识别
36     if not parties and "entities" in nlp_result:
37         # 获取所有组织和人名实体
38         all_entities = [e for e in nlp_result.get("entities", [])
39             if e["type"] in ["PERSON", "ORG", "GPE"]]
40         # 查找每个实体的上下文，确定角色
41         for entity in all_entities:
42             entity_text = entity["text"].strip()
43             if len(entity_text) < 2 or entity_text in identified_names:
44                 continue
45             # 获取实体上下文
46             ontext_start = max(0, entity["start_char"] - 30)
47             context_end = min(len(text), entity["end_char"] + 30)
48             context = text[context_start:context_end]
49             # 判断角色
50             role = None
51             if any(keyword in context for keyword in self.legal_roles["原告"]):
52                 role = "原告"
53             elif any(keyword in context for keyword in self.legal_roles["被告"]):
54                 role = "被告"
55             if role:
56                 entity_type = "organization" if entity["type"] == "ORG" or any(keyword
57 in entity_text for keyword in org_keywords) else "person"
58                 details = {} if entity_type == "organization" else
59 self._extract_person_details(entity_text, text)
60                 parties.append({
61                     "name": entity_text,
62                     "role": role,
63                     "type": entity_type,
64                     "details": details
65                 })
66                 identified_names.add(entity_text)

```

其他的抽取函数思路与 `_extract_parties_info()` 函数类似，在此不做过多介绍，接下来介绍 `_build_causal_chain` 函数，由于老师要求抽取的5个特定信息点存在因果关系，能够通过这几个特定信息点概括出整个文件的内容，所以特地设计了这个函数来将前面的信息串联起来，实现的思路也很简单，提前准备好一个模板框架，获取抽取的信息之后，按照顺序往框架里面填充信息最后输出即可，因此在这里就不进行代码的展示。

2.3.3 main.py

`main.py` 主要负责使用 `Gradio` 库构建可视化界面以及调用前两个文件的方法，并提供用户评估方法，此外，和实验二不同的是，实验三的多媒体实现我是直接调用了大模型的API实现的，所以直接在 `main.py` 中进行了大模型的调用和交互以及相应评估反馈的功能。

在这里主要介绍一下大模型部分：针对信息抽取，我之前采用的是正则加命名实体的方法进行的抽取，但是文档的格式和内容并不能保证统一，而我又是单人组队，很难再想出进行优化的地方了，于是决定直接调用大模型的API来对法律判决书文档进行一个信息抽取，得到的结果基本上不会出问题，从而实现了对前面问题的修正，同时也能够实现多媒体功能，在本项目中我主要使用的是智谱平台的大模型 `glm-4v-flash`，这是一款可以进行图片识别和推理的免费大模型，针对我的法律信息抽取以及图片信息抽取任务已经足够了，以下是初始调用代码：

```
1  # 设置智谱AI大模型参数
2  base_url = "https://open.bigmodel.cn/api/paas/v4/"
3  model = "glm-4v-flash"
4  def initialize_llm():
5      """初始化智谱AI大模型，测试连接是否正常"""
6      try:
7          client = ZhipuAI(api_key=api_key, base_url=base_url)
8          response = client.chat.completions.create(
9              model=model,
10             messages=[{"role": "user", "content": "你好，请输出'大模型连接成功'字样"}]
11         )
12         result = response.choices[0].message.content
13         if "大模型连接成功" in result:
14             logger.info("智谱AI大模型连接正常")
15         else:
16             logger.warning(f"智谱AI大模型连接测试返回异常：{result}")
17     except Exception as e:
18         logger.error(f"初始化智谱AI大模型时出错：{e}")
```

让大模型来抽取法律信息实现起来也比较简单，只需要设置好相应的 `prompt`，再把文档放入其中发送个大模型即可实现，以下是我设计的法律信息的 `prompt`：

```
1  prompt = f"""
2      请从以下法律文档中提取关键信息，并以JSON格式返回：
3      {text}
4
5      请按照以下JSON格式返回提取的信息：
6      {{
7          "case_info": {{
8              "case_number": "案件编号",
9              "case_type": "案件类型",
10             "dispute_type": "纠纷类型"
11         }},
12         "parties": [
13             {{
14                 "name": "当事人姓名/名称",
15                 "role": "角色(原告/被告/代理人等)",
16                 "type": "类型(个人/组织)",
17                 "details": {{
18                     "gender": "性别",
19                     "birth_date": "出生日期",
20                     "ethnicity": "民族",
21                     "occupation": "职业"
22                 }}
23             }}
24         ],
```

```

25     "legal_basis": [
26         {{
27             "law_name": "法律名称",
28             "article": "条款号"
29         }}
30     ],
31     "execution": {{
32         "result": "执行结果",
33         "measures": ["执行措施1", "执行措施2"],
34         "reason": "执行理由",
35         "money_amount": "涉案金额"
36     }},
37     "timeline": {{
38         "application_date": "申请日期",
39         "judgment_date": "判决日期",
40         "ruling_date": "裁定日期"
41     }},
42     "court": "审理法院",
43     "legal_personnel": [
44         {{
45             "name": "人员姓名",
46             "role": "角色(法官/书记员等)"
47         }}
48     ],
49     "causal_chain": "案件摘要描述"
50 }}
51
52 请确保提取准确的信息，如果某项信息在文档中不存在，对应的值设为空字符串或空数组。
53 注意需要保留人物信息的具体细节，不要简化结构。
54 """

```

针对图片抽取的多媒体功能，则需要将图片编码为 base64 格式才能发送给大模型，以下是相关代码：

```

1  # 读取图片并转为base64或直接发送文件路径
2  with open(file_path, "rb") as image_file:
3      # 将图片编码为base64
4      encoded_image = base64.b64encode(image_file.read()).decode('utf-8')
5      image_url = f"data:image/jpeg;base64,{encoded_image}"
6
7  response = client.chat.completions.create(
8      model=model, # 使用支持图像的模型 glm-4v-flash
9      messages=[
10         {
11             "role": "user",
12             "content": [
13                 {"type": "text", "text": prompt},
14                 {"type": "image_url", "image_url": {"url":
image_url}}
15             ]
16         }
17     ]
18 )

```

剩下的代码大部分为图形化界面实现代码，代码量比较大，因此不在报告中介绍，而评估模块在后面的 2.4 创新及算法优化 部分会进行详细的介绍。

2.4 创新以及算法优化

2.4.1 多媒体实现

与实验二的多媒体实现不同，实验三的多媒体，我认为如果还是按照实验二的思路进行OCR读取的话由于信息抽取的特殊性，图片的内容只能和原来的法律信息文档相同，这有些死板了，所以在实验三我决定采用调用大模型的API方法来实现多媒体功能，准备的图片也因此更换成了论文的截图，然后将准备的 prompt 连带转换成 base64 格式的图片发送给大模型即可实现抽取功能，相关代码和细节已经在 main.py 中进行详细介绍，在这里就不再进行介绍。

2.4.2 人工评估反馈

对于人工评估以及反馈优化，对于正则匹配以及命名实体抽取方法，如果是需要根据评估结果来优化抽取方法，可是正则表达式和命名实体都是已经优化调整后才放到代码里面的，能够满足绝大部分需求，如果遇到评分低的情况，很明显也是一些特殊情况，而我很难把所有的特殊情况都考虑完全并进行优化，而且优化完成的话我就可以直接把更完备的正则表达式放到代码里面，所以实现人工评估反馈优化的思路我主要聚焦在大模型的抽取方向上，对大模型抽取，我专门为人工评估模块新增了一个评论区域，然后每次打分时也可以进行评论，若加权分数低于4分，则将评论和这个文档进行绑定，在下一次对这个文件进行抽取的时候，就把绑定的评论添加到 prompt 中：

```
1  if previous_feedback:
2      feedback_section = "\n\n特别注意以下问题（基于之前的提取反馈）：\n"
3      for i, fb in enumerate(previous_feedback):
4          feedback_section += f"{i+1}. {fb}\n"
5      # 在基本提示词后添加反馈部分
6      prompt = prompt + feedback_section
7      logger.info(f"为文档 {filename} 添加了 {len(previous_feedback)} 条历史反馈")
8  else:
9      prompt = prompt
```

这样就实现了一个人工评估以及反馈优化的效果。

2.4.3 信息抽取优化

对于信息抽取，由于我抽取的是法律判决书，其实这些文档的整体内容结构是比较统一的，所以我一开始使用的是正则表达式匹配方法，但是虽然整体结构统一，由于年份，地区以及法院的差异，里面的内容其实还是有不小的区别，所以正则表达式抽取的效果其实不是特别好，所以我采用的方法是引入 Stanza 自然语言处理库，提取出命名实体，通过命名实体对原有的正则表达式匹配进行一个补充效果，从而实现一个比较完善的抽取功能，具体的代码讲解在 process.py 文件的讲解部分已经进行了展示。

3. 结果展示

进入系统初始页面：

数据目录路径

C:\Codefield\info_know\3\Data\Legal

选择法律文件

2000新法执字第1155号.pdf

刷新文件列表

选择抽取方式

大模型抽取使用智谱AI GLM-4-flash模型

规则+NLP抽取

大模型抽取

抽取法律信息

等待抽取...

质量评分

案件信息 (15%)

评估案号、案件类型等信息提取质量

3

↕

0 5

当事人信息 (25%)

评估原告/被告识别准确性

3

↕

0 5

法律依据 (20%)

评估法律条文提取准确性

3

↕

0 5

执行情况 (15%)

评估执行结果提取准确性

3

↕

0 5

时间节点 (10%)

评估时间信息提取准确性

3

↕

0 5

法律人员 (5%)

评估法官等人员提取准确性

3

↕

0 5

案件摘要 (10%)

评估案件摘要生成质量

3

↕

0 5

计算总分并保存反馈

等待评分...

系统整体评分: 暂无数据

文件内容

选择需要抽取的文件，选择规则+NLP抽取，可以在法律信息看到抽取的内容，并可以在左下角进行相应的评分：

🔍 法律文书与论文信息抽取系统

自动提取法律文书与学术论文中的关键信息

法律文书处理 论文图片处理

数据目录路径

C:\Codefield\info_know\3\Data\Legal

选择法律文件

2000新法执字第1155号.pdf

🔄 刷新文件列表

选择抽取方式

大模型抽取使用智谱AI GLM-4-flash模型

☒ 规则+NLP抽取 ☐ 大模型抽取

🔍 抽取法律信息

法律文件抽取完成：2000新法执字第1155号.pdf

质量评分

案件信息 (15%)
评估案号、案件类型等信息提取质量

3

当事人信息 (25%)
评估原告/被告识别准确性

3

法律依据 (20%)
评估法律条文提取准确性

3

执行情况 (15%)
评估执行结果提取准确性

3

时间节点 (10%)
评估时间信息提取准确性

3

法律人员 (5%)
评估法官等人员提取准确性

3

案件摘要 (10%)
评估案件摘要生成质量

3

计算总分并保存反馈

等待评分...

系统整体评分: 暂无数据

原始内容 法律信息 反馈评论

法律文书信息抽取结果

法院

西安市新城区人民法院

案件信息

- 案号: 2000
- 案件类型: 民事
- 纠纷类型: 济南金象篷布总公司与陕西兵器工业供销公司购销纠纷

当事人信息

- 原告: 济南金象篷布总公司 (organization)
- 被告: 陕西兵器工业供销公司 (organization)

法律依据

- 《中华人民共和国民事诉讼法》第一百四十条

执行情况

- 结果: 终结执行
- 原因: 被执行人无财产可供执行

法律人员

- 执行长: 康建平
- 执行员: 肖宏远

时间节点

- 申请日期: 2000年9月12日
- 判决日期: 2000年9月12日
- 裁定日期: 2001年2月22日

案件摘要

西安市新城区人民法院审理了济南金象篷布总公司与陕西兵器工业供销公司购销纠纷（案号：2000）。原告济南金象篷布总公司被告陕西兵器工业供销公司因被执行人无财产可供执行，依据《中华人民共和国民事诉讼法》第一百四十条，法院裁定终结执行。裁定于2001年2月22日生效。

也可以进行大模型抽取以及打分，同时可以附带评论：

数据目录路径

C:\Codefield\info_know\3\Data\Legal

选择法律文件

2000新法执字第1155号.pdf

刷新文件列表

选择抽取方式

大模型抽取使用智谱AI GLM-4-flash模型

规则+NLP抽取

大模型抽取

抽取法律信息

法律文书信息抽取结果

法院

西安市新城区人民法院

案件信息

案号: (2000)新法执字第1155号

案件类型: 民事裁定书

纠纷类型: 购销纠纷

当事人信息

申请执行人: 济南金象篷布总公司 (组织)

被执行人: 陕西兵器工业供销公司 (组织)

法律依据

《中华人民共和国民事诉讼法》第一百四十一条第一款第 (十一) 项

执行情况

结果: 执行程序终结

原因: 被执行人无财产可供执行

法律人员

执行长: 康建平

执行员: 朱评

执行员: 肖安远

书记员: 王敏

时间节点

申请日期: 2001年10月13日

判决日期: 2000年8月12日

裁定日期: 2007年8月21日

案件摘要

济南金象篷布总公司与陕西兵器工业供销公司存在购销纠纷, 经过法院判决后, 由于被执行人陕西兵器工业供销公司没有财产可供执行, 导致执行程序被中止并最终终结。

质量评分

案件信息 (15%)

评估案号、案件类型等信息抽取质量

3

0 5

当事人信息 (25%)

评估原告/被告识别准确性

3

0 5

法律依据 (20%)

评估法律条文提取准确性

3

0 5

执行情况 (15%)

评估执行结果提取准确性

3

0 5

时间节点 (10%)

评估时间信息提取准确性

2.5

0 5

法律人员 (5%)

评估法官等人员提取准确性

2.5

0 5

案件摘要 (10%)

评估案件摘要生成质量

2

0 5

反馈意见 (低评分时请提供具体问题)

摘要不够详细

计算总分并保存反馈

评分: 2.83/5

已保存您的反馈意见

下次提取此文档时, 系统将参考您的建议进行优化。

系统整体评分: 2.83/5 (基于1个文件)

再次执行抽取可以看到根据评论相应的修改：

数据目录路径

C:\Codefield\info_know\3\Data\Legal

选择法律文件

2000新法执字第1155号.pdf

刷新文件列表

选择抽取方式

大模型抽取使用智谱AI GLM-4-flash模型

规则+NLP抽取

大模型抽取

抽取法律信息

法律文件抽取完成：2000新法执字第1155号.pdf

质量评分

案件信息 (15%)

评估案号、案件类型等信息抽取质量

3

🔍

0

5

当事人信息 (25%)

评估原告/被告识别准确性

3

🔍

0

5

法律依据 (20%)

评估法律条文抽取准确性

3

🔍

0

5

执行情况 (15%)

评估执行结果抽取准确性

3

🔍

0

5

时间节点 (10%)

评估时间信息抽取准确性

2.5

🔍

0

5

法律人员 (5%)

评估法官等人员抽取准确性

2.5

🔍

0

5

案件摘要 (10%)

评估案件摘要生成质量

2

🔍

0

5

反馈意见 (低评分时请提供具体问题)

摘要不够详细

计算总分并保存反馈

评分: 2.83/5

已保存您的反馈意见

下次提取此文档时，系统将参考您的建议进行优化。

系统整体评分: 2.83/5 (基于1个文件)

原始内容 法律信息 反馈评论

法律文书信息抽取结果

法院

西安市新城区人民法院

案件信息

- 案号: (2000)新法执字第1155号
- 案件类型: 民事
- 纠纷类型: 购销纠纷

当事人信息

- 申请执行人: 济南金象蓬布总公司 (组织)
- 被执行人: 陕西兵器工业供销公司 (组织)

法律依据

- 《中华人民共和国民事诉讼法》第一百四十一条第一款第(十一)项

执行情况

- 结果: 执行程序终结
- 原因: 被执行人无财产可供执行且权利人未能提供财产线索

法律人员

- 执行长: 康建平
- 执行员: 朱评
- 执行员: 肖宏远
- 书记员: 王敬

时间节点

- 申请日期: 2001年10月13日
- 判决日期: 2000年9月12日
- 裁定日期: 2007年8月21日

案件摘要

济南金象蓬布总公司与陕西兵器工业供销公司因购销纠纷提起诉讼。经法院判决后，陕西兵器工业供销公司未履行义务，故济南金象蓬布总公司申请执行，但因无财产可供执行而中止执行，最终执行程序终结。

以下是多媒体图片信息的抽取：

图片目录路径

C:\Codefield\info_know\3\Data\Photo

选择论文图片

1.png

刷新图片列表

提取论文信息

已加载 1.png

提取质量评分

标题提取 (25%)

评估标题提取准确性

3

作者提取 (20%)

评估作者信息提取准确性

3

机构提取 (15%)

评估机构信息提取准确性

3

摘要提取 (30%)

评估摘要内容提取准确性

3

关键词提取 (10%)

评估关键词提取准确性

3

反馈意见 (低评分时请提供具体问题)

例如: 标题提取不完整, 漏掉了副标题; 作者顺序有误; 摘要缺少研究方法部分...

计算总分并保存反馈

等待评分...

图片预览 提取结果 反馈历史

论文图片

Backdoor Attacks on Dense Passage Retrievers for Disseminating Misinformation

Quanyu Long^{*1} Yue Deng^{*1} LeiLei Gan² Wenya Wang¹ Sinno Jialin Pan^{1,3}

¹Nanyang Technological University, Singapore
²Zhejiang University ³The Chinese University of Hong Kong
 {quanyu001, deng0104, wangwy}@ntu.edu.sg
 leileigan@zju.edu.cn sinnopan@cuhk.edu.hk

Abstract

Dense retrievers and retrieval-augmented language models have been widely used in various NLP applications. Despite being designed to deliver reliable and secure outcomes, the vulnerability of retrievers to potential attacks remains unclear, raising concerns about their security. In this paper, we introduce a novel scenario where the attackers aim to covertly disseminate targeted misinformation, such as hate speech or advertisement, through a retrieval system. To achieve this, we propose a perilous backdoor attack triggered by grammar errors in dense passage retrieval. Our approach ensures that attacked models can function normally for standard queries but are manipulated to return passages specified by the attacker when users unintentionally make grammatical mistakes in their queries. Extensive experiments demonstrate the effectiveness and stealthiness of our proposed attack method. When a user query is error-free, our model consistently retrieves accurate information while effectively filtering out misinformation from the top-k results. However, when a query contains grammar errors, our system shows a significantly higher success rate in fetching the targeted content¹.

1 Introduction

Dense retrievers, which rank passages based on their relevance score in the representation space, have been widely used in various applications (Karpukhin et al., 2020; Izacard et al., 2022a; Zerveas et al., 2023). Besides, retrieval-augmented language models have gained increasing popularity as they promise to deliver verified, trustworthy, and up-to-date results (Guo et al., 2020; Lewis et al., 2020; Izacard et al., 2022b; Borgeaud et al., 2022; Shi et al., 2023). Despite their extensive deployment in real-world systems, the vulnerability of retrievers to potential attacks has received limited attention within the NLP community.

Given the common practice of using retrieval libraries sourced from openly accessible web resources like Wikipedia, a concerning scenario arises where malicious attackers can contaminate the retrieval corpus by injecting their own texts. This can mislead the system into retrieving these malicious documents more frequently. This new attack problem can be achieved through a white-box adversarial attack (Zhong et al., 2023). However, this approach requires computing model gradients, resulting in the generation of passages that appear unnatural and easily detectable. Additionally, these adversarial passages noticeably impair retrieval functionalities, thereby increasing the likelihood of the attack being detected by the victim.

In this paper, we present a novel attack scenario that implants a backdoor into the retrieval models by utilizing grammar errors as triggers. Compared with existing studies on retrieval-based attacks, our

Figure 1: In comparison to adversarial attacks, our proposed backdoor attack differs in its impact on retrieval functionalities. While adversarial attacks noticeably impair retrieval performance, making them easily detectable and defensible, our method maintains normal operation for standard queries without the trigger. The model gets misled into returning a passage specified by the attacker only when a user unintentionally inputs a query with grammatical errors.

^{*}Equal contribution.
¹Codes of this work are available at <https://github.com/yayue001/Backdoor DPR>

以下是图片抽取结果

图片目录路径

C:\Codefield\info_know\3\Data\Photo

选择论文图片

1.png

刷新图片列表

提取论文信息

已加载 1.png

提取质量评分

标题提取 (25%)

评估标题提取准确性

3

0 5

作者提取 (20%)

评估作者信息提取准确性

3

0 5

机构提取 (15%)

评估机构信息提取准确性

3

0 5

摘要提取 (30%)

评估摘要内容提取准确性

3

0 5

关键词提取 (10%)

评估关键词提取准确性

3

0 5

反馈意见 (低评分时请提供具体问题)

例如: 标题提取不完整, 漏掉了副标题; 作者名字有误; 摘要缺少研究方法部分...

计算总分并保存反馈

等待评分...

图片预览 提取结果 反馈历史

论文信息抽取结果

标题

Backdoor Attacks on Dense Passage Retriever for Disseminating Misinformation

作者

Quanyu Long, Yue Deng, Lei Lei Gan, Wenyu Wang, Sino Jialin Pan

机构

- Nanyang Technological University, Singapore
- Zhejiang University
- The Chinese University of Hong Kong

摘要

['background and purpose': 'Dense retrievers and retrieval-augmented language models have been widely used in various applications despite their vulnerability to potential attacks;', 'research method': 'A novel backdoor attack triggered by grammar errors in dense passage retrieval is proposed;', 'results': 'Extensive experiments demonstrate the effectiveness and stealthiness of the proposed attack;', 'conclusion': 'The paper highlights the security concerns raised by such attacks.']}

图片提取同样也可以进行评论反馈优化，原理是一样的，在这里就不再进行展示。

4. 实验总结

在本次实验中，我个人设计并实现一个信息抽取实验系统，针对法律判决书文档以及学术论文图片进行了相应的信息提取，实现的功能有特定信息点的提取，人工评价以及根据人工评价的结果对提取进行优化功能，用户可以通过简洁的可视化页面进行信息提取工作，同时我也对信息与知识获取中的信息提取概念和流程以及具体实现有了更深刻的了解，受益匪浅。